Programming and Problem Solving

NovoNex Delta



TOPIC 01 • AGENDA

Introduction to Programming & Compilers

- Programming
- Hardware
- Compilation
- Binaries
- Execution

Introduction to Programming & Compilers

- **Programming:** Writing precise instructions that the CPU executes to solve problems.
- **Hardware:** Silicon CPU and physical memory (RAM) process binary electric states.
- **Compilation:** GCC translates human-readable high-level C code (.c) into machine code.
- **Binaries:** CPUs can only directly execute native machine binary instructions (0s and 1s).
- **Execution:** OS loads compiled machine code into RAM for ultra-fast CPU execution.

💡 Standard GCC Command: gcc -Wall main.c -o program

✏️ Live Trace & Explanation Board

[Teacher's Note: Architecture trace from high-level .c source to CPU execution]

From Source Code to Machine Execution:

1. Source File (.c):
   Human-readable C syntax statements and functions.

2. GCC Compilation Process:
   [Preprocessor] -> [Compiler] -> [Assembler] -> [Linker]

3. Executable Binary (.exe / ELF):
   Native 0s and 1s CPU instruction opcodes.

4. OS Loader & Physical RAM:
   Loads machine binary into memory; CPU executes at clock speed!

• C bridges high-level logic and bare-metal hardware control.

TOPIC 02 • AGENDA

C Program Structure & Syntax

- Headers
- Main
- Statements
- Blocks
- Comments

C Program Structure & Execution Flow

- **Headers:** Directives like #include <stdio.h> import standard I/O library prototypes.
- **Main:** Every C executable begins execution strictly at the mandatory int main().
- **Statements:** Instructions in C must terminate with a semicolon (;) to mark completion.

```c
// Mandatory C Skeleton
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0; // Signals OS success
}
```

💡 **Golden Rule: Execution always begins at main(), regardless of physical file position!**

✏️ Live Trace & Execution Flow

[Line-by-line execution walkthrough of main entry point]

Program Execution Anatomy:

1. Preprocessor (#include):
   Pastes standard header declarations into compilation unit.

2. Function Header (int main()):
   Defines return type (int) and program entry point.

3. Standard Function Call (printf):
   Transfers bytes to stdout; '\n' moves terminal cursor.

4. Return Statement (return 0):
   Returns status code 0 to operating system (Success!).

Blocks, Scopes & Documentation Syntax

- **Blocks:** Curly braces { } define code blocks and isolate local variable scopes.
- **Comments:** Single-line // and multi-line /* */ document logic and are ignored by compilers.
- **Statements:** Compound statements grouped within braces execute as a single atomic unit.

```
                    // Scope Block & Comments
int total = 100; // Global to block
{
    int local = 50; // Inner scope only
    total += local;
}
/* 'local' is destroyed after block! */
```

💡 Best Practice: Always use clear comments and consistent indentation for block scopes!

📦 Block Scope & Lifecycle Trace

[How curly braces govern memory allocation and lifetime]

1. Lexical Scope Isolation

- Variables declared inside { } exist only in that block
- Outer scopes cannot access variables in inner blocks
- Prevents variable name collisions in large programs

2. Semicolon & Comment Rules

- Semicolons terminate statements, NOT block braces { }
- Placing a semicolon right after 'if ();' creates an empty statement trap!
- Comments do not add any byte overhead to final binaries

TOPIC 03 • AGENDA

Variables, Data Types & Standard I/O

- Variables
- Types
- Specifiers
- Input
- Output

Variables, Data Types & Memory Allocation

- **Variables:** Named memory locations in physical RAM allocated to store program data.
- **Types:** Fundamental types set size: int (4B), float (4B), double (8B), char (1B).
- **Specifiers:** Format specifiers inform functions how to interpret raw binary bytes.

```
                    // Data Type Declarations
int count = 42;          // 4 Bytes: Whole numbers
float price = 19.99f;    // 4 Bytes: 6 decimal places
double pi = 3.14159265; // 8 Bytes: 15 decimal places
char grade = 'A';        // 1 Byte:  ASCII character
```

💡 **Memory Rule: sizeof(type) queries the exact byte footprint on the target hardware!**

🧠 RAM Memory Box Architecture

[Hardware memory allocation and bit representations]

Physical Memory Architecture:

Variable Storage in Hardware RAM:

Variable	Type	Size (B)	Memory Address
count	int	4 Bytes	0x7ffd5e20 (RAM)
price	float	4 Bytes	0x7ffd5e24 (RAM)
pi	double	8 Bytes	0x7ffd5e28 (RAM)
grade	char	1 Byte	0x7ffd5e30 (RAM)

- Variables map directly to contiguous memory cells
- Hexadecimal addresses locate each variable in RAM

Standard Console Input & Output Streams

- **Specifiers:** Tokens (%d, %f, %lf, %c) direct formatted printing and reading conversions.
- **Input:** scanf() captures user terminal keystrokes and requires the memory address (&).
- **Output:** printf() writes formatted character streams directly to standard console.

```
// Formatted Console Reading & Writing
int age; double gpa;
printf("Enter age and gpa: ");
scanf("%d %lf", &age, &gpa);
printf("Age = %d | GPA = %.2lf\n", age, gpa);
```

⚠ Critical Trap: Forgetting '&' in scanf() causes segmentation fault crashes!

⌨ Standard I/O Buffer Tracing

[How printf and scanf interact with stdin/stdout]

1. Why scanf Requires '&'

- C passes arguments by value (copies)
- To modify 'age', scanf must receive its address (&age)
- Without '&', scanf interprets age's value as an address!

2. Common Format Specifiers

- %d -> Signed decimal integer (int)
- %f -> Single-precision float
- %lf -> Double-precision float (in scanf)
- %c -> Single character (char)

TOPIC 04 • AGENDA

Operators, Identifiers, & Expression Evaluation

- Identifiers
- Arithmetic
- Relational
- Logical
- Precedence

Identifiers & Basic Operator Mechanics

- **Identifiers:** Programmer-defined variable and function names following strict naming rules.
- **Arithmetic:** Math operators: +, -, *, / (quotient), and % (modulo integer remainder).
- **Relational:** Comparison operators (==, !=, <, >, <=, >=) evaluate to 1 (True) or 0 (False).

```
// Arithmetic & Relational Evaluation
int a = 7, b = 2;
int quot = a / b;     // 3 (integer truncation!)
int rem = a % b;      // 1 (remainder)
int is_greater = a > b; // 1 (True)
```

⚠️ **Integer Division Trap: 7 / 2 yields 3! Use 7.0 / 2 to obtain 3.5 float result!**

✏️ Live Operator Evaluation Board

[Identifier rules & arithmetic truncation behavior]

Operator Rules & Identifiers:

1. Valid Identifier Rules:
   - Must start with letter (a-z, A-Z) or underscore (_)
   - Cannot start with a digit (e.g. 2var is INVALID)
   - Case-sensitive: 'Total' and 'total' are distinct!

2. Modulo Operator (%) Constraints:
   - Only works with integer operands (not float/double)
   - Useful for parity checks: (n % 2 == 0) -> Even!

3. Relational Truth Values:
   - In C, comparisons yield int 1 for TRUE and 0 for FALSE.

Logical Operators & Precedence Ladder

- **Logical:** Boolean operators (&&, ||, !) enable compound condition testing with short-circuiting.
- **Precedence:** Precedence ranking resolves operator priority in compound mathematical expressions.
- **Relational:** Relational evaluations bind tighter than logical tests but looser than arithmetic.

```
                    // Compound Logic & Precedence
int x = 5, y = 10;
// && has higher precedence than ||
int res = (x > 0) && (y < 20); // 1 (True)
int calc = x + y * 2;          // 25 (* precedes +)
```

💡 Golden Rule: Always use parentheses () to make operator precedence explicit!

📊 Graphical Operator Precedence Ladder

[Priority hierarchy & evaluation direction]

1. Parentheses: () [] -> . | Left to Right
2. Unary Operators: ++ -- ! sizeof & | Right to Left
3. Multiplicative: * / % | Left to Right
4. Additive: + - | Left to Right
5. Relational: < <= > >= | Left to Right
6. Equality: == != | Left to Right
7. Logical AND / OR: && then || | Left to Right
8. Assignment: = += -= *= | Right to Left

TOPIC 05 • AGENDA

Conditional Statements (if, else if, else) and Nested Logic

- Branching
- Conditions
- Alternatives
- Nesting
- Fallthrough

Conditional Statements: if, else if, else

- **Branching:** Alters sequential instruction flow based on runtime evaluation of conditions.
- **Conditions:** Expressions where 0 evaluates to False and any non-zero value evaluates to True.
- **Alternatives:** else if chains and trailing else blocks guarantee mutual exclusion across paths.

```c
// Multi-Way Branching: Grade Classifier
if (score >= 90) {
    printf("Grade: A\n");
} else if (score >= 80) {
    printf("Grade: B\n");
} else {
    printf("Grade: C\n");
}
```

⚠️ Danger: if (x = 5) assigns 5 (always TRUE!). Always use '==' for comparison checks!

✏️ Live Trace & Decision Trees

[Trace decision tree: Single if vs if-else vs else-if ladder]

Decision Logic Architecture:

1. Single if:
   [Condition True?] ──Yes──> [Execute Body]
   | No
   ▼ (Continue sequential flow)

2. Two-way if-else:
   [Condition True?] ──Yes──> [True Branch]
   | No
   └──────────> [False Branch]

3. Multi-way else-if Ladder:
   Tests conditions sequentially from top to bottom;
   first matching True block executes, rest skipped!

Nested Conditionals & Switch Logic

- **Nesting:** Embedding an if-else inside another conditional tests granular multi-tier logic.
- **Fallthrough:** In switch blocks, execution cascades into next cases unless halted with break;.
- **Alternatives:** The default: branch reliably captures any unexpected or unmatched inputs.

```
            // Multi-Way Decision: switch with break
switch (op) {
    case '+': res = a + b; break;
    case '-': res = a - b; break;
    case '*': res = a * b; break;
    default:  printf("Invalid operator\n");
}
```

💡 **Best Practice: Flatten deep nested conditionals using guard clauses and early exits!**

⚡ Decision Tree vs Switch Jump Table

[Architectural comparison of multi-branching]

1. Nested If-Else Decision Tree

- Checks conditions sequentially: O(N) comparisons
- Ideal for ranges (score >= 80 && score < 90)
- High nesting levels reduce code readability

2. Switch-Case Jump Table

- Compiler compiles into direct O(1) jump table
- Strictly tests discrete integer or char values
- Missing 'break;' triggers case fallthrough!

TOPIC 06 • AGENDA

Flow chart

- Flowcharts
- Symbols
- Processes
- Decisions
- Patterns

Programming and Problem Solving

Flowchart Shapes

Core Process & I/O Symbols

[Visual shapes and standard operations for program flow]



Terminator (Oval / Rounded Pill)

- **Purpose:** Defines the mandatory entry (Start) and exit (End / Stop) boundary terminals of program flow.
- `C: Entry at main() and exit at return 0;`



Process Box (Rectangle)

- **Purpose:** Represents computational tasks, arithmetic calculations, data transformations, and assignments.
- `C: sum = a + b;  |  count = 0;  |  x++;`



Input / Output (Parallelogram)

- **Purpose:** Represents data receiving operations from user input or printing results to output displays.
- `C: scanf("%d", &a);  |  printf("Result: %d", sum);`

💡 Golden Rule: Every flowchart must have exactly 1 Start and at least 1 End point!

Decision & Routing Symbols

[Branching conditions, connector junctions & directional flow]



Decision Box (Diamond)

- **Purpose:** Evaluates a boolean conditional question; splits flow into divergent True / False paths.
- `C: if (score >= 50)  |  while (i < n)`



Connector (Circle)

- **Purpose:** Joins multiple intersecting flow lines into a single point to prevent messy crossing lines.
- `C: Merge point of if-else branches, loop returns`



Flow Line / Arrow

- **Purpose:** Indicates the exact sequence order and directional path of instruction execution.
- `C: Sequential step-by-step instruction execution`

⚡ Key Rule: Decision diamonds must always branch into at least 2 labeled paths!

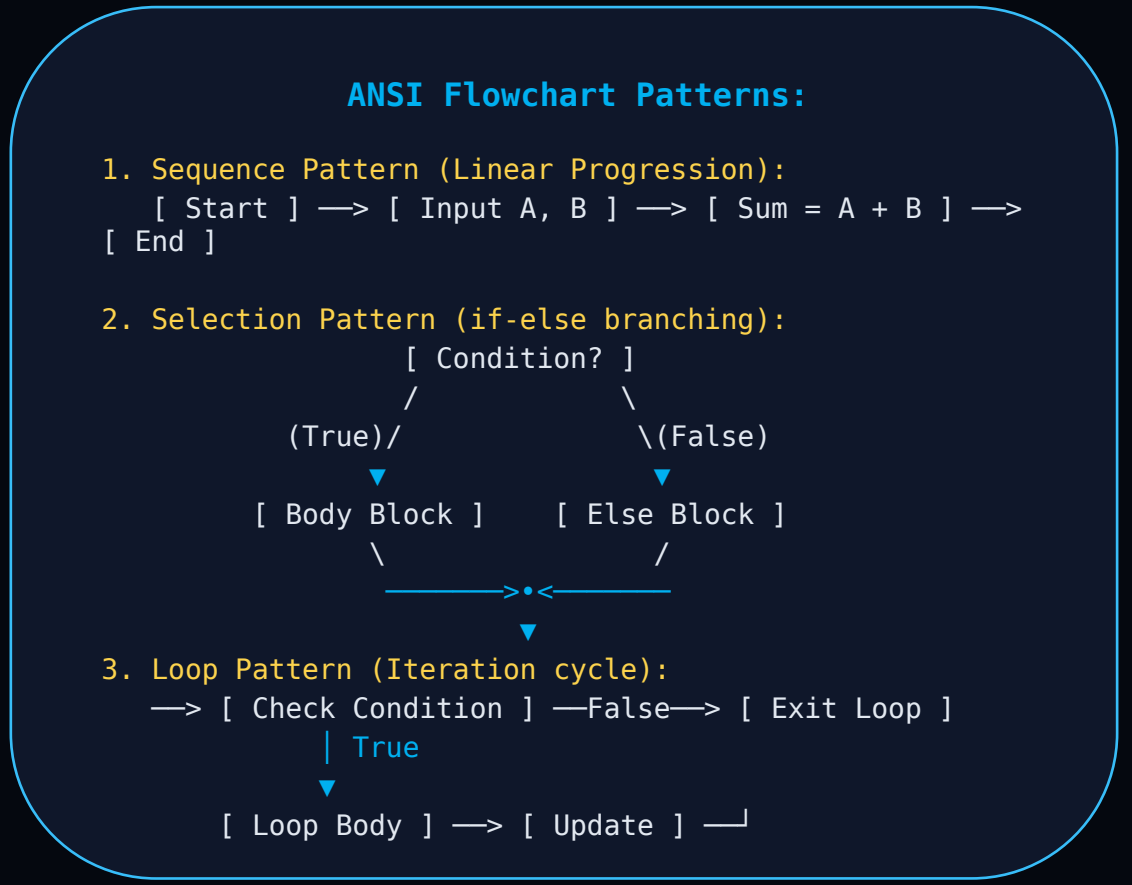
Flowchart Fundamentals & Logic Flow

- **Flowcharts:** Standardized visual blueprints representing algorithmic step-by-step program logic.
- **Purpose:** Clarifies complex logic, prevents syntax & logic bugs, and acts as an architectural guide.
- **Execution:** Control flows strictly along directed lines from the Start node to an End node.
- **Dry Running:** Enables step-by-step manual tracing of variables with test data prior to coding.
- **Golden Rules:** Must possess unambiguous pathways, a single entry terminal, and reachable exits.

💡 **Best Practice: Design and trace your flowchart logic on paper before writing actual C code!**

📐 Core Flowchart Control Patterns

[Visual representation of 3 fundamental algorithmic control structures]



TOPIC 07 • AGENDA

Iterative Statements (Loops) and Nested Loops

- Iteration
- Initialization
- Condition
- Update
- Nesting

Iterative Statements: for, while & do-while

- **Iteration:** Repeated execution of an instruction block until a stopping condition is met.
- **Initialization:** Counter variables are initialized once prior to entering the loop cycle.
- **Condition:** Boolean expression verified before each pass; loop halts immediately when False.
- **Update:** Loop counter is modified (e.g., i++) after every

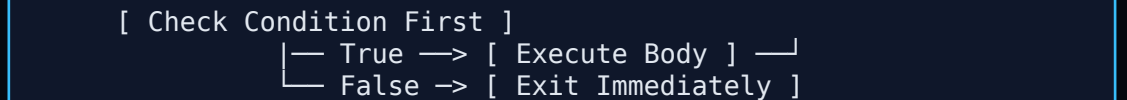
```
// Counter Controlled Loop (for)
for (int i = 0; i < 5; i++) {
    printf("%d ", i);  // Prints: 0 1 2 3 4
}
// Condition Controlled Loop (while)
int n = 10;
while (n > 0) { n /= 2; }
```

💡 Execution Order: 1. Init -> 2. Condition Check -> 3. Body Execute -> 4. Counter Update

🔄 while vs do-while Flow Comparison

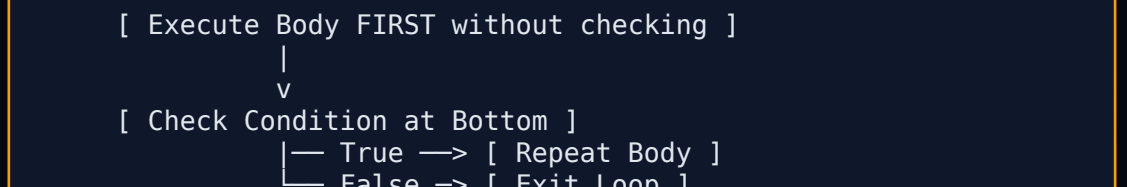
[Entry-controlled vs Exit-controlled architecture]

1. while Loop (Entry-Controlled Flow)



- May execute 0 times if condition starts FALSE!

2. do-while Loop (Exit-Controlled Flow)



- Guaranteed to execute at least 1 time!

Nested Loops & Grid Traversal

- **Nesting:** A loop placed entirely inside another loop creates multi-dimensional iteration cycles.
- **Iteration:** The inner loop runs completely from start to finish for every iteration of outer loop.
- **Update:** Outer loop updates row indices, while inner loop steps through column coordinates.

```
            // Pattern Example: 3 Rows x 4 Stars Grid
for (int r = 1; r <= 3; r++) {     // Outer controls Rows
    for (int c = 1; c <= 4; c++) { // Inner controls Columns
        printf("* ");
    }
    printf("\n");
}
```

💡 **Golden Rule: Outer loop always controls Rows (r); Inner loop controls Columns (c)!**

📐 Nested Loop Matrix Grid Traversal

[How 2 loops generate 2D grids and patterns]

Visual Execution Trace: 3 Rows x 4 Columns

```
Outer (r=1)  -> Inner (c=1..4):   *  *  *  *   [Newline]
Outer (r=2)  -> Inner (c=1..4):   *  *  *  *   [Newline]
Outer (r=3)  -> Inner (c=1..4):   *  *  *  *   [Newline]
```

Execution Step Trace:

- Outer loop runs 3 times (r = 1, 2, 3)
- For each r, inner loop runs 4 times (c = 1, 2, 3, 4)
- Total printf("* ") calls = 3 x 4 = 12 times!
- Total time complexity = O(Rows x Cols)

TOPIC 08 • AGENDA

1D Arrays: Memory Layout & Indexing

- Arrays
- Indexing
- Contiguity
- Addresses
- Bounds

1D Arrays: Definition & Zero-Based Indexing

- **Arrays:** Fixed-size collections storing elements of identical data type in contiguous memory.
- **Indexing:** Zero-based indexing (0 to N-1) grants direct, instant O(1) random access to elements.
- **Bounds:** C performs no runtime bounds checks; accessing beyond bounds leads to memory corruption.

```c
// 1D Array Declaration, Input & Output
int arr[5] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++) {
    printf("arr[%d] = %d\n", i, arr[i]);
}
```

⚠️ **Out of Bounds: Accessing arr[5] in a size-5 array leads to undefined behavior / garbage!**

🧠 1D Array Contiguous RAM Layout

[Physical memory allocation in hardware]

Visual RAM Memory Cells: int arr[5] = {10, 20, 30, 40, 50}



- Base Address: Address of arr[0] = 1000 (&arr[0] or arr)
- Formula: Address(arr[i]) = Base + (i x sizeof(type))
  Example: Address(arr[3]) = 1000 + (3 x 4) = 1012
- Contiguous: Zero gaps between elements in RAM!

1D Arrays: Memory Layout & Pointer Offsets

- **Contiguity:** Array elements occupy adjacent, unbroken bytes in physical RAM without any gaps.
- **Addresses:** Memory location computed via hardware formula: Address(arr[i]) = Base + (i * sizeof(type)).
- **Bounds:** Array name acts as a constant pointer to the base address of the first element (&arr[0]).

```
// Pointer Arithmetic & Array Addressing
int arr[3] = {100, 200, 300};
printf("Base Address: %p\n", (void*)arr);
printf("arr[1] Addr:  %p\n", (void*)&arr[1]);
// Address increases by sizeof(int) = 4 bytes!
```

💡 Golden Rule: Array indexing arr[i] is syntactically equivalent to *(arr + i)!

📐 Hardware Pointer Offset Mechanics

[How the CPU computes array element memory addresses]

1. Linear Address Calculation

- Base Address = starting byte of arr[0] in RAM
- Offset formula = Base + (index * byte_size)
- Multiplication is done in hardware using bit shifts
- Gives constant O(1) random access time!

2. The Danger of Index Out-of-Bounds

- arr[-1] or arr[5] reads adjacent stack memory
- Can overwrite loop counters, return addresses
- Major source of critical security exploits!

TOPIC 09 • AGENDA

2D Arrays (Matrices): Grid & Memory Mapping

- Matrices
- Rows
- Columns
- Traversal
- Linearization

2D Arrays: Grid Coordinates & Matrix Model

- **Matrices:** Two-dimensional arrays structured conceptually as a table grid of Rows and Columns.
- **Rows:** The primary dimension index referencing the horizontal row partition of matrix data.
- **Columns:** The secondary dimension index accessing individual element cells along horizontal rows.

```
// 2D Array Declaration & Grid Setup
int mat[2][3] = {
    {1, 2, 3},  // Row 0
    {4, 5, 6}   // Row 1
};
printf("Cell (0,1): %d\n", mat[0][1]); // 2
```

💡 Convention: In mat[R][C], first index R specifies Row, second index C specifies Column!

2D Coordinate Grid Matrix

[Logical table visualization of rows and columns]

Conceptual 2D Grid (2 Rows x 3 Cols):

```
          Col 0     Col 1     Col 2
Row 0:   [ (0,0) ] [ (0,1) ] [ (0,2) ]
Row 1:   [ (1,0) ] [ (1,1) ] [ (1,2) ]
```

Logical Cell Values:

- mat[0][0] = 1, mat[0][1] = 2, mat[0][2] = 3
- mat[1][0] = 4, mat[1][1] = 5, mat[1][2] = 6
- Total cells = ROWS x COLS = 2 x 3 = 6 integers!

2D Arrays: Traversal & Row-Major RAM Mapping

- **Traversal:** Nested loops navigate 2D arrays: outer loop steps rows (i), inner loop steps columns (j).
- **Linearization:** Physical RAM is linear; C flattens 2D matrices into 1D memory using Row-Major order.
- **Rows:** Row 0 elements are stored consecutively in RAM, followed immediately by Row 1 elements.

```c
// 2D Array Traversal & Address Tracing
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        printf("mat[%d][%d]=%d ", i, j, mat[i][j]);
    }
    printf("\n");
}
```

💡 **Linear Offset Formula: Address(mat[i][j]) = Base + (i x COLS + j) x sizeof(type)**

🧠 2D Grid to Row-Major RAM Mapping

[Hardware linearization of 2D matrices]

Row-Major Memory Order in Physical RAM:

1. Conceptual 2D Grid (2 Rows x 3 Cols):

```
            Col 0   Col 1   Col 2
   Row 0:   [  1  ] [  2  ] [  3  ]
   Row 1:   [  4  ] [  5  ] [  6  ]
```

2. Physical Linear RAM Storage (Row-Major):

```
   [ 1 ] [ 2 ] [ 3 ]   -->   [ 4 ] [ 5 ] [ 6 ]
   └──── Row 0 ────┘         └──── Row 1 ────┘
```

- Row 0 is stored consecutively, followed immediately by Row 1
- Row-wise iteration maximizes CPU cache hit rates!

TOPIC 10 • AGENDA

Error finding in C code

- Syntax
- Linking
- Runtime
- Logic
- Debugging

Spot the Bug: Syntax & Input/Output Traps

- **Syntax:** Grammatical mistakes violating C rules like missing semicolons or case errors (Printf).
- **Linking:** Unresolved symbol references occurring when function declarations have no definitions.
- **Debugging:** Enabling aggressive compiler warnings (-Wall -Wextra) exposes syntax bugs instantly.

```
            // Snippet A: Reading integer from user
int age; scanf("%d", age);           // BUG #1: Missing &
// Snippet B: Score output
int score = 95
Printf("Score: %d\n", score);       // BUGS #2 & #3: Semicolon & Printf
// Snippet C: Float printing
float gpa = 3.85; printf("%d", gpa); // BUG #4: %d with float
```

🔍 **Student Challenge: Identify the 4 errors above and explain the compiler errors!**

✏️ Live Trace & Bug Fix Board

[Classroom Activity: Ask students to spot errors, circle them live, and write corrected code]

Bug Diagnosis & Fix Guide:

Bug #1 Fix:

- `scanf("%d", &age); // Pass memory address with &!`

Bug #2 & #3 Fix:

- `int score = 95;  // Semicolon required after statement!`
- `printf("Score..."); // C is case-sensitive: 'printf', not 'Printf'!`

Bug #4 Fix:

- `printf("GPA: %.2f\n", gpa); // Use %f (or %lf) for floating point!`

- Always compile with 'gcc -Wall -Wextra' to catch these early!

Spot the Bug: Logic, Loops & Array Traps

- **Logic:** Using assignment '=' instead of equality '==' in conditionals always evaluates to True.
- **Runtime:** Off-by-one array boundary violations (i <= N) read or overwrite garbage memory.
- **Debugging:** Tracing variable states across loop boundary iterations quickly exposes off-by-one flaws.

```
            // Snippet D: Conditional Security Check
int is_admin = 0;
if (is_admin = 1) {       // BUG #5: Assignment '=' vs '=='
    printf("Admin Access Granted!\n");
}
// Snippet E: Array Traversal & Printing
int arr[5] = {10, 20, 30, 40, 50};
for (int i = 0; i <= 5; ) { // BUGS #6 & #7: Bounds <= 5 &
missing i++
    printf("%d ", arr[i]);
}
```

🔍 **Student Challenge: These compile without error but give wrong results! Why?**

✏️ Live Trace & Bug Fix Board

[Classroom Activity: Trace condition truth table, index bounds, and float cast]

Subtle Logic Bug Fixes:

Bug #5 Fix (Assignment vs Equality):
• if (is_admin == 1) // '=' assigns 1 (always TRUE); use '=='!

Bug #6 & #7 Fix (Array Bounds & Infinite Loop):
• for (int i = 0; i < 5; i++) // '< 5' prevents arr[5] crash, 'i++' updates!

General Debugging Practice:
• Trace loops using a dry-run iteration table
• Check boundary edges (index 0 and index N-1)
• Treat compiler warnings as fatal errors (-Werror)!

Thank You

Happy Coding with NovoNex Delta!

